

Reflection on Design Choices for the Smart Greenhouse System

Throughout the development of the Smart Greenhouse Sensor Simulation, several key design choices were made to ensure the system was robust, strictly adhered to the assignment constraints, and met the principles of Object-Oriented programming.

Data Types and Class Structure

I selected primitive data types strategically to match the nature of the data. For the Timestamp class, integers (int) were used for all temporal fields (day, month, year, hour, minute) because these are inherently whole numbers and allow for straightforward mathematical validation (e.g., ensuring hours remain between 0 and 23). For the SensorReading class, the value field was designated as a double to accommodate precision data, such as temperatures of 27.4 Celsius. The system was structured using composition, where the SensorReading class contains a Timestamp object. This separation of concerns ensures that the logic for validating dates remains isolated from the logic for validating sensor data.

Menu System Approach

The menu system was designed using a procedural, modular approach wrapped in a continuous while loop within Main.java. To prevent deeply nested switch statements and reduce clutter, I abstracted the menu selections into a flexible helper method (getValidInputInRange). This method handles both out-of-bounds integers and InputMismatchExceptions, ensuring the program never crashes due to erratic user input.

Challenges Encountered

One of the most significant challenges was managing arrays dynamically without the use of Java's ArrayList or Collections frameworks. Because standard Java arrays have fixed sizes, I had to write custom array-resizing logic (the append method in SensorData.java) which creates a new array of $N+1$ size and copies elements over when adding data. Similarly, deleting a record required shifting elements and rewriting the entire CSV file. While this is computationally heavy $O(n)$ time complexity, it strictly satisfied the assignment's core requirement to manually iterate over standard arrays.

Future Improvements

If given more time, I would improve the efficiency of the file reading and writing processes. Currently, deleting a record requires overwriting the entire CSV file, which could become a performance bottleneck with datasets containing millions of rows. Additionally, I would unify the file-path handling logic to ensure the Logger and SensorData classes always write to the exact same dynamic directory relative to the executable, preventing potential FileNotFoundException exceptions across different operating systems.